

LangChain & Agentic AI – From Basics to Advanced demo-16 Notes

=====

```
from pydantic import BaseModel, Field

class Student(BaseModel):
    name:str
    age:int | None = Field(default=None, description="if age of student")
    course:str = Field(description = "coursename")
```

```
s = Student(
    name="durga",
    course="Python"
)
```

To get dictionary from Pydantic Object:

```
d = s.model_dump() #dict
```

```
=====
```

```
from pydantic import BaseModel, Field
```

```
class Product(BaseModel):
```

```
    name: str = Field(
        description="Name of the product"
    )
```

```
    category: str = Field(
        description="Category of the product"
    )
```

```
    price: float = Field(
        description="Price of the product"
    )
```

```
    in_stock: bool | None = Field(
        default = False,
```

```
description="Whether the product is available"  
)
```

```
p = Product(name = "Iphone18",category = "smartphone",price  
= 300000,in_stock=True)  
print(p)  
print(p.name)  
print(p.model_dump()) #dict  
print(type(p.model_dump())) #dict  
print(p.model_dump_json()) #json  
print(type(p.model_dump_json())) #json
```

Note:

```
product -->Pydantic Object  
product.name -->Individual Field  
product.model_dump() --->Python dictionary  
product.model_dump_json() --->JSON string
```

LangChain with Structured output:

It means asking the model to return information according to predefined schema instead of returning arbitrary free-form text.

Ravi age 22 years studying python

name=Ravi

age=22

course=Python

LangChain defined the method: with_structured_output()

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(Student)
```

Create a model wrapper that should return output matching
Student schema

=====

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel
```

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
    course: str
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
structured_llm = llm.with_structured_output(Student)
```

```
result = structured_llm.invoke(
```

```
"Ravi is 22 years old and he is learning Python"
)
print(type(result))
print(result)
print(result.name)
print(result.age)
print(result.course)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
```

```
class Student(BaseModel):
```

```
    name: str = Field(
        description="Name of the student"
    )
```

```
    age: int = Field(
```

```
        description="Age of the student"  
    )
```

```
course: str = Field(  
    description="Course or technology the student is learning"  
)
```

```
llm = ChatOpenAI(  
    model="gpt-5.6",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(Student)
```

```
text = input("Enter student information: ")
```

```
result = structured_llm.invoke(text)
```

```
print("\nComplete Result:")
```

```
print(result)
```

```
print("\nName:")  
print(result.name)
```

```
print("\nAge:")  
print(result.age)
```

```
print("\nCourse:")  
print(result.course)
```

E:\lcclasses>py test.py

Enter student information: Shiva is a wonderful student
learning langchain and his dob is 18-04-1992

Complete Result:

name='Shiva' age=34 course='LangChain'

Name:

Shiva

Age:

34

Course:

LangChain

=====

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class Employee(BaseModel):
```

```
    name: str = Field(
```

```
        description="Name of the employee"
```

```
    )
```

```
    technology: str = Field(
```

```
        description="Primary technology of the employee"
    )
```

```
    experience: int = Field(
        description="Years of professional experience"
    )
```

```
llm = ChatOpenAI(
    model="gpt-5.6",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
structured_llm = llm.with_structured_output(Employee)
```

```
text = input("Enter employee information: ")
```

```
employee = structured_llm.invoke(text)
```

```
print("\nEmployee Details")
```

```
print("-----")
```

```
print("Name      :", employee.name)
print("Technology :", employee.technology)
print("Experience :", employee.experience)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class Product(BaseModel):
```

```
    name: str = Field(
        description="Name of the product"
    )
```

```
    category: str = Field(
        description="Product category"
    )
```

```
    price: float = Field(
```

```
        description="Price of the product"
    )

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(Product)

text = input("Enter product information: ")

product = structured_llm.invoke(text)

print("\nProduct Object:")
print(product)

print("\nIndividual Fields:")
print("Name    :", product.name)
print("Category :", product.category)
```

```
print("Price   :", product.price)
```

```
print("\nAs Dictionary:")
```

```
print(product.model_dump())
```

```
print("\nAs JSON:")
```

```
print(product.model_dump_json())
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class Book(BaseModel):
```

```
    title: str = Field(
        description="Title of the book"
    )
```

```
    author: str = Field(
```

```
        description="Author of the book"  
    )
```

```
category: str = Field(  
    description="Category or genre of the book"  
)
```

```
llm = ChatOpenAI(  
    model="gpt-5.6",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(Book)
```

```
result = structured_llm.invoke(  
    "Give information about Atomic Habits."  
)
```

```
print(result)
```

```
print("\nTitle:", result.title)
print("Author:", result.author)
print("Category:", result.category)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
```

```
class Person(BaseModel):
```

```
    name: str = Field(
        description="Name of the person"
    )
```

```
    profession: str = Field(
        description="Profession of the person"
    )
```

```
email: str | None = Field(  
    default=None,  
    description="Email address if available"  
)
```

```
llm = ChatOpenAI(  
    model="gpt-5.6",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(Person)
```

```
result = structured_llm.invoke(  
    "Ravi is a Python developer."  
)
```

```
print(result)
```

```
print("Name:", result.name)
```

```
print("Profession:", result.profession)
```



```
print("Email:", result.email)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class Person(BaseModel):
```

```
    name: str = Field(
```

```
        description="Name of the person"
```

```
)
```

```
    profession: str = Field(
```

```
        description="Profession of the person"
```

```
)
```

```
    experience: int | None = Field(
```

```
        default=None,
```

```
        description="Number of years of professional experience"
```

```
)
```

```
skills: list[str] = Field(  
    description="Important professional skills"  
)
```

```
llm = ChatOpenAI(  
    model="gpt-5.6",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(Person)
```

```
text = input(  
    "Enter information about a person: "  
)
```

```
result = structured_llm.invoke(text)
```

```
print("\n-----")
```

```
print("COMPLETE PYDANTIC OBJECT")
```

```
print("-----")
```

```
print(result)
```

```
print("\n-----")
```

```
print("OBJECT TYPE")
```

```
print("-----")
```

```
print(type(result))
```

```
print("\n-----")
```

```
print("INDIVIDUAL FIELDS")
```

```
print("-----")
```

```
print("Name      :", result.name)
```

```
print("Profession :", result.profession)
```

```
print("Experience :", result.experience)
```

```
print("Skills    :", result.skills)
```

```
print("\n-----")
```

```
print("PYTHON DICTIONARY")
```

```
print("-----")
```

```
print(result.model_dump())
```

```
print("\n-----")
```

```
print("JSON STRING")
```

```
print("-----")
```

```
print(result.model_dump_json())
```

```
=====
```

MCQ 1

Which output is more application-friendly when individual values are required?

- A. Random paragraph
- B. Structured data
- C. Long story
- D. AIMessage metadata

Ans: B

MCQ 2

JSON stands for:

- A. Java Standard Object Network
- B. JavaScript Object Notation
- C. JSON Structured Object Name
- D. Java Syntax Object Notation

Ans: B

MCQ 3

Which Pydantic class do we commonly inherit from?

- A. AIMessage
- B. BaseModel
- C. ChatOpenAI
- D. PromptTemplate

Ans: B

MCQ 4

What does this mean?

age: int

- A. Age is an AI model
- B. Age should be an integer
- C. Age is a prompt
- D. Age is always optional

Ans: B

MCQ 5

Which represents an optional email field with a default of no value?

A.

email: str

B.

email = str

C.

email: str | None = None

D.

optional(email)

Ans: C

MCQ 6

What does this do?

```
structured_llm = llm.with_structured_output(Student)
```

A. Deletes the LLM

B. Creates a wrapper that returns data following the Student schema

C. Converts Student to PDF

D. Creates conversation memory

Ans: B

=====

Notes:

Point 1

Plain text is excellent for humans.

Structured data is often better for applications.

Point 2

Unstructured:

Ravi is 22 and learning Python.

Structured:

name = Ravi

age = 22

course = Python

Point 3

JSON is a common data-interchange format.

Point 4

JSON and Python dictionaries look similar, but they are not the same concept.

Point 5

A schema defines what structure our application expects.

Point 6

A field represents one piece of data.

Point 7

Fields can have different types:

str

int

float

bool

list[str]

Point 8

Pydantic helps us define data models and validate data.

Point 9

We commonly create Pydantic models using:

`BaseModel`

Point 10

Fields may be required or optional.

Point 11

`Field()` can provide descriptions and other field information.

Point 12

`model_dump()`

gives us a Python dictionary.

Point 13

`model_dump_json()`

gives us JSON text.

Point 14

LangChain provides:

`with_structured_output()`

for schema-based structured model output.

Point 15

With a Pydantic schema, we can work directly with a Pydantic object.

=====